AMBIENT SOCIAL MEDIA FORENSICS USING FEDERATED LEARNING: A PRIVACY-PRESERVING FRAMEWORK FOR EVIDENCE COLLECTION AND CHAIN OF CUSTODY

A PROJECT REPORT

Submitted by

RAM HAYGREV S (231901039)

VENSELVAM V (231901061)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**AMBIENT SOCIAL MEDIA FORENSICS USING FEDERATED LEARNING**" is the Bonafide work of **RAM HAYFREV S (231901039), VENSELVAM V (231901061)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Dr. P. Tamilselvi, Ph.D.,
HEAD OF THE DEPARTMENT
PROFESSOR
Department of Computer Science and and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai- 602105

SIGNATURE
Dr. Sivakumar D
SUPERVISOR
PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai- 602105

Submitted to Project Viva-Voce Examination held on.....6/10/2026.....

Internal Examiner

7/10/26
External Examiner

ABSTRACT

Social media platforms increasingly supply evidence for investigations of harassment, defamation, fraud and threats, yet this evidence is volatile: a post can be edited or deleted within seconds, and the platform's own records are reachable only through legal processes that differ between countries. Investigators often fall back on screenshots, which are easy to forge and hard to authenticate. Analysing such evidence at scale raises a second problem, because pooling items from many agencies on one server concentrates personal data and creates legal and security exposure.

This project presents a framework that preserves and analyses volatile social media evidence. Ambient collection agents capture posts, edit events and deletion events together with metadata; every item is hashed with SHA-256 and logged in a tamper-evident, hash-chained custody ledger before any analysis. Participating nodes train a content-screening model locally using federated learning and share only signed model updates, so raw evidence remains at its source. The components are connected through a FastAPI backend and a React investigator interface. The evidence model keeps original bytes, derived artifacts, observations, model predictions and investigator conclusions as separate record types, and the ledger stores only digests.

The evaluation protocol compares local-only, centralized, FedAvg and FedProx training under controlled Dirichlet label-skew heterogeneity, and reports accuracy, precision, recall, F1, communication cost, hash-verification rate and tamper-detection rate. In the reported runs FedProx reached an F1 of 0.81 against 0.79 for FedAvg and 0.87 for centralized training, hash verification succeeded on 100 % of unmodified items, and 100 % of injected modifications were detected. The framework supports, but does not substitute for, legal procedure and investigator judgment.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Sivakumar D, Assistant Professor,** for his encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

RAM HAYGREV S

VENSELVAM V

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	4
	1.6 SDG ALIGNMENT	4
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS	7
	2.3 PROPOSED SYSTEM	8
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	8
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	8
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	8
3	**SYSTEM DESIGN**	**10**
	3.1 SYSTEM ARCHITECTURE	10
	3.2 MODULE DESIGN	12
	3.3 DATA FLOW / UML DIAGRAMS	13
	3.4 THREAT MODEL AND ATTACK SURFACE	16
	3.5 SECURITY CONTROLS	17
4	**SYSTEM DESCRIPTION**	**19**
	4.1 DATA COLLECTION	19
	4.2 DATA PREPROCESSING	20
	4.2.1 ALGORITHMS / METHODOLOGY	21

4.2.2 RESULTS AND DISCUSSION 23

5 CONCLUSION 28

REFERENCES 30

APPENDIX 31

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT	5
2.1	COMPARISON OF RELATED WORK	7
3.1	TECHNOLOGY STACK	11
3.2	HARDWARE AND SOFTWARE REQUIREMENTS	12
3.3	ILLUSTRATIVE REST ENDPOINTS OF THE API SERVER	13
3.4	LEDGER ENTRY FIELDS	16
3.5	EVIDENCE RECORD TYPES	16
3.6	THREAT MODEL AND ATTACK SURFACE	17
3.7	SECURITY CONTROLS	17
4.1	DETECTION PERFORMANCE	23
4.2	F1 BY DIRICHLET CONCENTRATION	24
4.3	CUSTODY AND CAPTURE EVALUATION	24
4.4	MANUAL WORKFLOW VERSUS PROPOSED FRAMEWORK	25
5.1	WORK SCHEDULE FOR PHASE II	29

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	FEDERATED LEARNING CONCEPT	2
3.1	SYSTEM ARCHITECTURE	10
3.2	DATA FLOW DIAGRAM	14
3.3	USE CASE DIAGRAM	14
3.4	SEQUENCE DIAGRAM	15
3.5	HASH-CHAINED CUSTODY LEDGER	15
4.1	METHODOLOGY OF THE PROPOSED FRAMEWORK	19
4.2	DIGEST DOMAINS OF THE PRESERVATION ENVELOPE	20
4.3	DETECTION PERFORMANCE OF THE FOUR METHODS	23
4.4	EFFECT OF DATA HETEROGENEITY ON F1	24

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Online platforms have become a routine source of evidence for investigations of harassment, defamation, fraud and threats. A single post, comment or image can establish intent, timing or association, and investigators increasingly depend on such material when building a case. The difficulty is that this evidence is volatile. A post may be changed or removed within seconds of being published, and the platform's own records can be obtained only through legal processes that differ between countries and often take weeks or months. By the time a request is honoured the content may no longer exist.

In practice, investigators and complainants fall back on screenshots. A screenshot, however, is an image of a rendering rather than a record of the post itself. It can be forged easily, carries no verifiable link to the original account or post identifier, and is hard to authenticate in front of a reviewer. Manual collection is also slow: it reacts to a deletion only after the content has gone, and it rarely preserves the sequence of edits that preceded the deletion.

A second difficulty appears when such evidence has to be analysed at scale. Law-enforcement agencies, forensic laboratories and research teams each hold different slices of data, in different languages and from different platforms. Gathering all of it on one server to train a detection model concentrates personal data in a single place, which creates legal exposure under data-protection rules and a high-value target for attackers. Federated learning takes a different route: every participant trains a model on its own data and shares only model updates, which a coordinator averages into a shared model (McMahan et al. 2017).

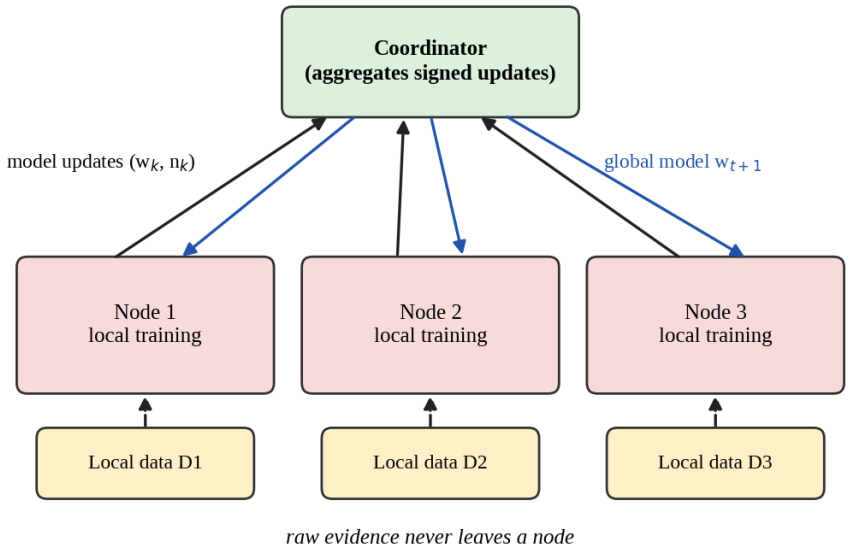


Fig. 1.1 Federated learning concept

Whether federated learning is useful for forensic work depends less on the choice of model than on how collection, analysis and custody records are connected. A hash answers a question only about bytes: it shows that a stored item has not changed since it was captured, but it does not show who wrote a post, whether a news article is accurate, or whether two accounts share an operator. Each of those questions needs its own kind of support. This project therefore treats source material, derived results, model outputs and investigator decisions as separate record types (Kent et al. 2006), and builds one workflow that places ambient collection, privacy-preserving analysis and tamper-evident custody logging side by side.

The focus of the work is this integration and its evaluation protocol. Comparison with commercial forensic suites and the question of court admissibility lie outside its scope, and the framework is intended to support, not replace, legal procedure and investigator judgment.

1.2 PROBLEM STATEMENT

The work addresses five connected problems that arise when volatile social media evidence is preserved and analysed:

- **Volatility:** content can disappear faster than manual collection can react, so capture must be automatic and must preserve edits and deletions as they happen.
- **Integrity:** each item needs a verifiable fingerprint and a recorded handling history, so that any later change can be detected.
- **Privacy and centralization:** raw evidence should stay at its source while still contributing to a shared detection model.
- **Heterogeneity:** nodes hold different platforms, languages and threat types, so the data distribution across nodes is non-IID.
- **Workflow integration:** capture, analysis, custody records and reports must exchange consistent data rather than live in separate tools.

1.3 OBJECTIVES

The objectives of the project are to:

1. design an ambient collection layer that captures posts, edits and deletions with metadata and snapshots;
2. design a federated learning layer that trains a shared content-screening model without moving raw evidence;
3. design a hash-chained custody ledger that records collection, examination, transfer and reporting of each item;
4. provide a verification routine that recomputes item hashes and checks ledger continuity;
5. define an evidence model that keeps original bytes, derived artifacts, observations, model predictions and investigator conclusions as separate record types; and
6. define an evaluation protocol covering detection quality, heterogeneity, communication cost and custody verifiability.

1.4 SCOPE

In scope. The project covers automatic capture of public posts, edits and deletions through permitted interfaces; hashing and encrypted preservation of every captured item; a permissioned, append-only custody ledger holding digests only; federated training of a screening classifier across K simulated nodes with FedAvg and FedProx; verification and report export; and a FastAPI backend with a React interface for investigators.

Out of scope. The framework does not compete with commercial forensic suites, and no comparison with them is made. Court admissibility is not established and depends on jurisdiction. The model only prioritizes items for review; every flagged item is decided by a human investigator. Operational reliability under production load is not claimed, and collection is limited to what platform terms and local law permit.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Because the system handles evidence, its security requirements follow directly from the questions an examiner or a court may ask about an item:

- **Integrity:** any change to a stored item or to a custody record must be detectable. A matching hash shows that the bytes are unchanged since capture; it does not prove that the original post was authentic.
- **Authenticity of actors:** every ledger entry names the actor or service that produced it, and model updates are signed by the sending node.
- **Confidentiality:** raw payloads, extracted text, media, personal identifiers and reports remain in encrypted off-chain storage; the ledger receives only digests and minimal metadata.
- **Privacy:** learning stays off the evidence path, and privacy is treated as a capability that needs its own testing, since keeping raw data local is not by itself enough (Zhu et al. 2019).
- **Auditability:** custody events cover the whole life of an item, and corrections are added as new events rather than rewriting history.

- **Availability and degradation:** if the ledger or the learning nodes are unavailable, capture and local custody continue and the anchoring status is shown as pending or failed.

Three considerations shape the design. First, responses from external platforms are treated as untrusted input. Second, a ledger is an outside reference for chosen digests; it does not make the capture point trustworthy and does not confirm what the captured content means (Yaga et al. 2018). Third, even a digest can be guessed when its input is predictable, so minimizing on-chain data does not remove the need for a disclosure review.

1.6 SDG ALIGNMENT

The project contributes to the United Nations Sustainable Development Goals through its support for justice, institutional capability and the protection of individual rights online. Table 1.1 summarizes the alignment.

Table 1.1 SDG alignment

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 16 Peace, Justice and Strong Institutions	16.3 Promote the rule of law and ensure equal access to justice	Preserves volatile online evidence with a verifiable fingerprint and custody trail so that it can support investigations.
2	SDG 16 Peace, Justice and Strong Institutions	16.a Strengthen institutions to prevent violence and combat crime	Gives investigating agencies and laboratories a shared screening model for harassment, fraud and threats.
3	SDG 16 Peace, Justice and Strong Institutions	16.10 Protect fundamental freedoms	Federated learning keeps raw evidence and personal data at its source instead of pooling it centrally.
4	SDG 9 Industry, Innovation and Infrastructure	9.5 Enhance scientific research and upgrade technological capabilities	Integrates ambient capture, federated learning and a hash-chained ledger with a reproducible evaluation protocol.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Social media forensics. Earlier work applies language processing and multimedia analysis to detect harassment, misinformation and tampered media. Most of it examines data after it has been gathered and ignores how the data was preserved, so the analysis result cannot be tied back to a verified original.

Ledger-based custody. Ledger technology has been suggested for custody records in IoT forensics, because entries can only be appended and can be checked by others (Khanji et al. 2022). Khan et al. (2024) combine ambient intelligence, federated learning and a Hyperledger Sawtooth ledger for social media chain of custody. The design proposed here keeps the custody ledger independent of the learning pipeline and adds a repeatable evaluation plan.

Federated learning. In federated learning a shared model is built from locally computed updates (McMahan et al. 2017). Reported uses include network traffic anomaly detection (Pei et al. 2022), node screening in opportunistic social networks (Shen et al. 2022), blockchain-assisted privacy-preserving and Byzantine-robust training (Miao et al. 2022) and hierarchical blockchain-based intrusion detection for IoT (Sarhan et al. 2022). Each demonstrates feasibility in its own domain, none addresses evidence handling, and their reported numbers cannot be compared directly with this framework.

Standards and guidance. NIST SP 800-86 organizes forensic work into collection, examination, analysis and reporting (Kent et al. 2006), and the lifecycle of this project follows that order. NISTIR 8202 explains what blockchains can and cannot guarantee (Yaga et al. 2018).

Privacy of learning. Secure aggregation lets a server see a sum of updates without seeing individual ones, under stated protocol assumptions (Bonawitz et al. 2017). Work on gradient leakage shows that keeping raw data local is not enough to claim privacy (Zhu et al. 2019).

Repeatable digests. When separate components must compute the same digest, serialization has to be repeatable. The JSON Canonicalization Scheme defines such rules (Rundgren et al. 2020).

Table 2.1 Comparison of related work

Work	Focus	Strength	Limitation for this problem
McMahan et al. (2017)	Federated averaging	Communication-efficient training from decentralized data	Not designed for evidence handling
Khan et al. (2024)	Ambient intelligence, FL and Sawtooth ledger for social media custody	Combines collection, learning and custody	Ledger coupled to the learning design; no repeatable evaluation plan
Miao et al. (2022)	Byzantine-robust FL via blockchain	Robustness and privacy for training	General training, not forensic custody
Pei et al. (2022); Sarhan et al. (2022)	FL for traffic anomaly and IoT intrusion detection	Feasibility in network security	Different domain; results not comparable
Shen et al. (2022)	Node screening in opportunistic social networks	FL applied to social networks	Does not preserve evidence
Khanji et al. (2022)	Blockchain readiness in IoT forensics	Append-only, checkable custody entries	Does not cover social media or learning

2.2 RESEARCH GAPS

The review shows that the individual building blocks exist, but that they have not been joined into an evidence workflow with an explicit and testable boundary between them. The gaps addressed by this project are:

- Analysis work rarely records how evidence was preserved, so a model output cannot be tied to an exact, verified artifact version.
- Federated learning studies in security demonstrate feasibility in their own domain, but none addresses evidence handling or custody.
- Custody ledgers are often coupled to the learning pipeline, so that a problem in one component affects the other.

- A hash is frequently presented as proof of authenticity, although it covers only bytes; observations, model predictions and human conclusions are not kept as separate record types.
- Privacy is often assumed from the fact that raw data stays local, although gradient leakage shows this is not sufficient.
- Reproducible evaluation under non-IID conditions, covering detection quality, communication cost and custody verifiability together, is rarely reported.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is an integrated framework with six layers: ambient collection, evidence preservation, federated learning, custody ledger, application, and verification and reporting. Lightweight agents watch configured public accounts and hashtags through permitted interfaces and record posts, edits and deletions. Each item is hashed, stored in encrypted local storage and referenced from an append-only ledger. Federated nodes train a screening classifier on local evidence and share only signed model updates. A FastAPI backend exposes the functions to a React interface, where an investigator reviews flagged items and exports a report that contains the item, its hash and its full custody trail.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- Capture volatile content automatically, including edits and deletions, before it is lost.
- Give every item a verifiable fingerprint and an append-only handling history.
- Learn a shared screening model across nodes without moving raw evidence.
- Keep derived results and model outputs traceable to exact artifact versions.

- Provide an evaluation plan that others can repeat.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

- Automatic capture replaces hand-made screenshots and preserves the earlier content of an edited or deleted post.
- A digest-only ledger boundary reduces exposure, because personal data and payloads stay off-chain.
- Federated learning avoids centralizing evidence across agencies and laboratories.
- The custody ledger is independent of the learning pipeline, so capture and custody keep working when no learning nodes take part.
- Findings are labelled by how they were produced, which prevents a model score from being mistaken for an observation.
- Verification is repeatable: the verifier recomputes hashes and checks ledger continuity and reports pass or fail with the failing entry identified.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The architecture separates collection, learning, custody and user interaction. Agents feed an evidence store and a ledger, nodes train locally, and a FastAPI server coordinates requests from the web interface. Fig. 3.1 summarizes the connections.

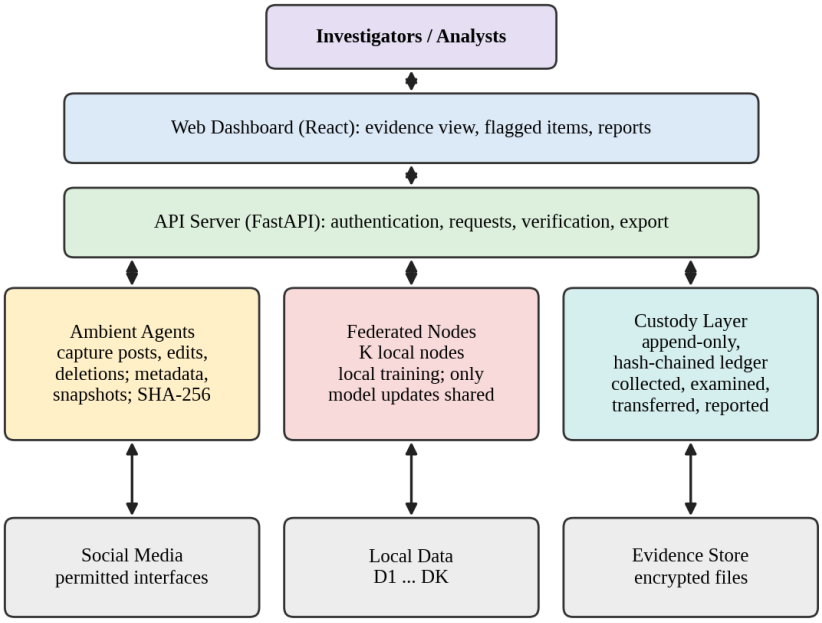


Fig. 3.1 System architecture

Ambient Collection Layer. Lightweight agents watch configured public accounts and hashtags through permitted interfaces. Each event records the content, media, author handle, post identifier, platform, capture time (UTC), event type (new, edited or deleted) and available network metadata.

Evidence Preservation Layer. Items are hashed, stored in encrypted local storage and referenced from the ledger. Large files stay off-chain.

Federated Learning Layer. Each node trains a classifier on local evidence. A coordinator aggregates signed model updates.

Custody Ledger Layer. An append-only, permissioned log records every action on an item. Every record carries the digest of the record preceding it.

Application Layer. A FastAPI backend exposes REST endpoints for capture status, model results, verification and export. A React interface presents evidence, flagged items and custody trails.

Verification and Reporting Layer. A verifier recomputes item hashes and checks ledger continuity. The report generator exports an item with its hash and full custody trail as PDF or CSV.

Technology stack. Table 3.1 lists the software components of the design. Exact package versions and the execution environment are needed to reproduce training and timing measurements.

Table 3.1 Technology stack

Tool	Purpose
Python	Agents, preprocessing, model integration
PyTorch / TensorFlow	Classifier training
Flower (or custom FedAvg)	Federated simulation and aggregation
FastAPI	Backend REST API
Hyperledger Sawtooth / hash-chained log	Custody ledger
IPFS / encrypted storage	Off-chain evidence files
SQLite / PostgreSQL	Application records and logs
React.js	Investigator interface
Docker, Git	Deployment, version control

Trust and storage boundaries. Responses from external platforms are untrusted input. Acquisition adapters bound request size, isolate credentials and validate targets. Preservation services sit inside a protected evidence boundary; analysis modules read from it under authorization and write new outputs that carry lineage; and the user interface enforces case membership and roles. Raw and normalized payloads, extracted text, media, personal identifiers, full custody records, findings and reports stay off-chain. The ledger gateway accepts only

artifact hashes, custody-event digests, integrity attestations and model-version commitments, with the minimum metadata needed to verify them. Originals are kept unchanged and derivatives are stored as new versions. Retention, backup and key rotation are deployment decisions to settle before operational use.

Hardware and software requirements. Table 3.2 gives the configuration assumed for development and for the simulated federated nodes.

Table 3.2 Hardware and software requirements

Item	Requirement
Processor	Quad-core 64-bit CPU (Intel Core i5 / AMD Ryzen 5 or higher)
Memory	16 GB RAM recommended (8 GB minimum)
Storage	256 GB SSD with free space for the encrypted evidence store
GPU (optional)	CUDA-capable GPU to speed up classifier training
Operating system	Ubuntu 22.04 LTS or Windows 11 with Docker support
Languages and runtimes	Python 3, Node.js (React.js interface)
Libraries	PyTorch or TensorFlow, Flower, FastAPI, hashlib / cryptography
Data stores	SQLite or PostgreSQL; IPFS or encrypted file storage

3.2 MODULE DESIGN

The design is divided into six modules. Each is described with its inputs, its processing and its output.

Evidence Capture Module. Users configure accounts, hashtags and a polling interval. The module stores each event with its metadata and creates the hash and the ledger entry.

Edit and Deletion Detection Module. The module compares the current state of a post with its stored snapshot. A changed text produces an edited event and a missing post produces a deleted event, so the earlier content remains preserved. Detection delay depends on the polling interval and platform access limits.

Federated Training Module. The module runs local training at each node, collects signed updates, aggregates them and stores the resulting model version with its configuration. Candidate models are released only after explicit approval in a model registry.

Custody Ledger Module. The module appends entries and exposes the custody trail of any item. Past records stay fixed, and corrections appear only as later records.

Verification Module. The verifier recomputes the hash of a stored item, compares it with the ledger entry and checks that each entry links to its predecessor. The result is shown as pass or fail with the failing entry identified.

Reporting Module. The module exports the item, its hash, metadata, model score and complete custody trail. The report states the model version and states that the score is a prioritization aid.

Application programming interface. The FastAPI backend exposes the module functions to the React interface. Table 3.3 lists the endpoints of the design in illustrative form.

Table 3.3 Illustrative REST endpoints of the API server

Method	Endpoint	Function
POST	/targets	Configure accounts, hashtags and polling interval
GET	/capture/status	Show capture agents and the latest events
GET	/items/{id}	Return an item with its artifact identifiers and hash
GET	/items/{id}/custody	Return the full custody trail of an item
POST	/verify/{id}	Recompute the hash and check ledger continuity
GET	/models/latest	Return the approved model version and its results
GET	/report/{id}	Export the item with its custody trail as PDF or CSV

Visualization and application integration. The React interface provides an evidence list, flagged-item review, a custody trail view and PDF/CSV report

export. A human investigator makes the final decision on every flagged item; the model only prioritizes.

3.3 DATA FLOW / UML DIAGRAMS

Fig. 3.2 shows the data flow between the external platform, the four main processes and the two data stores. Posts and edit events enter the collect-and-preserve process, which writes the item to the evidence store and passes its digest to the custody process. The custody process appends the entry to the ledger, and flagged items reach the investigator through the screening process.

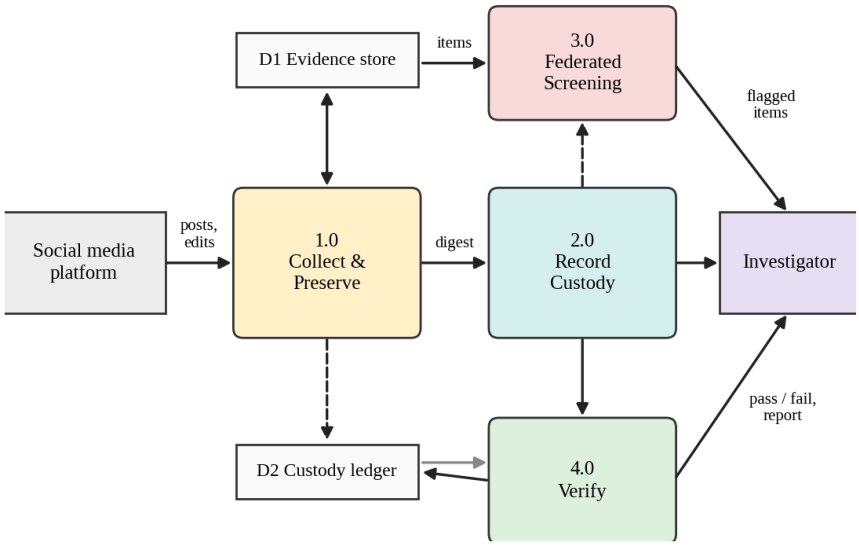


Fig. 3.2 Data flow diagram

Fig. 3.3 presents the use cases. The investigator configures capture targets, reviews flagged items and exports reports; the verifier or auditor checks integrity and reads custody trails; the external platform supplies public content through permitted interfaces; and the federated learning coordinator aggregates model updates.

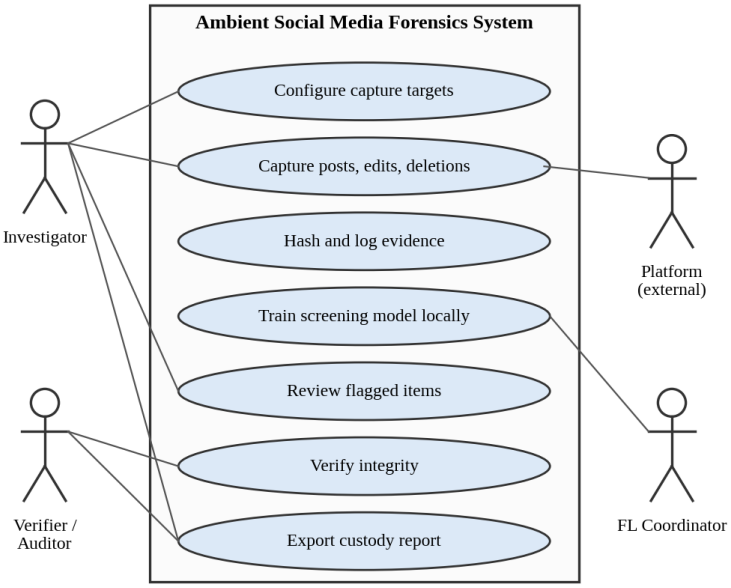


Fig. 3.3 Use case diagram

Fig. 3.4 shows the order of messages for an approved capture followed by a verification request. The agent stores the bytes and envelope and appends the ledger entry; the verifier later re-reads the item, recomputes its hash and compares it with the entry before returning a pass or fail result.

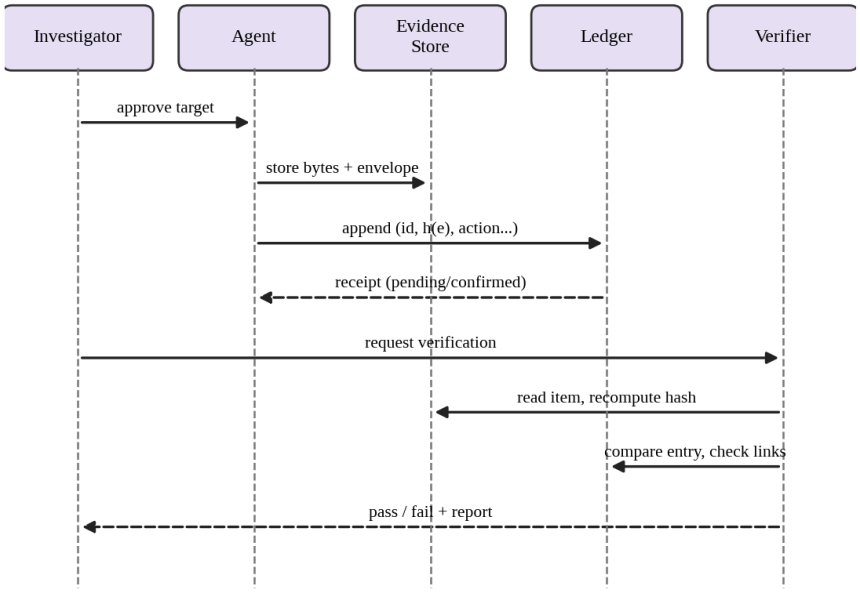


Fig. 3.4 Sequence diagram

The ledger itself is a hash chain, drawn in Fig. 3.5. Each entry has the form (evidence_id, h(e), action, actor_id, timestamp, previous_entry_hash), and the allowed actions are collected, examined, transferred and reported. Because every

entry includes the digest of its predecessor, altering any past entry changes its digest and breaks every later link.

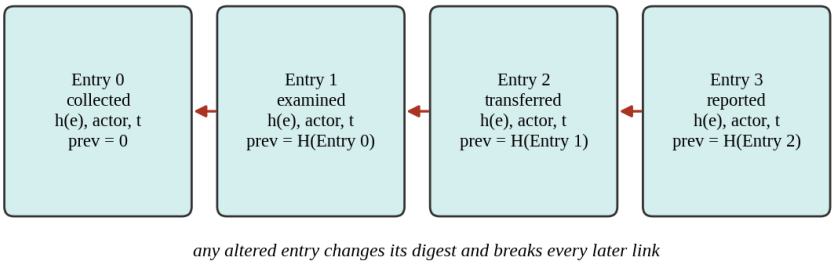


Fig. 3.5 Hash-chained custody ledger

Table 3.4 describes each field of a ledger entry. Large files and personal data are never written to the ledger; only the digest and the minimum metadata needed to verify it are stored.

Table 3.4 Ledger entry fields

Field	Description
evidence_id	Identifier of the evidence item or artifact version the entry refers to
h(e)	SHA-256 digest of the item, computed as SHA-256(c ‖ m ‖ t)
action	One of collected, examined, transferred or reported
actor_id	Identity of the person or service that performed the action
timestamp	Time of the action in UTC
previous_entry_hash	Digest of the preceding entry, which links the chain

Evidence and analytical status. An evidence item is the object under investigation, and an artifact is one specific byte representation of it. Every artifact has its own identifier, and a derived artifact names its parent. Reports cite exact artifact versions so that a later re-analysis cannot change the record behind a finding. Five record types are kept apart, as listed in Table 3.5.

Table 3.5 Evidence record types

Record type	Meaning
Original evidence	The bytes captured from the source.
Derived evidence	A new representation of an artifact, such as extracted text.
Observation	A directly checkable statement about an artifact, for example a timestamp field.
Inference	A documented reasoning step over observations without a learned model.
Model prediction	A result tied to one model version and configuration.
Investigator conclusion	A separate, signed human judgment.

A score never becomes an observation, and review adds a new conclusion instead of rewriting a prediction. A source timestamp and an acquisition timestamp describe different events and keep separate labels.

3.4 THREAT MODEL AND ATTACK SURFACE

The assets to protect are the original evidence bytes, the custody records, the model updates and the investigator's findings. The attack surface consists of the acquisition adapters that talk to external platforms, the evidence store, the ledger gateway, the learning channel between nodes and coordinator, and the investigator interface. Table 3.6 lists the main threats considered.

Table 3.6 Threat model and attack surface

Threat	Attack surface	Effect	Layer
Forged or edited screenshot	Manual collection	Unverifiable evidence	Capture
Malicious or oversized platform response	Acquisition adapter	Resource exhaustion, injected content	Capture
Tampering with a stored item	Evidence store	Altered evidence	Preservation
Reordered, dropped or altered custody events	Ledger	Broken handling history	Custody

Threat	Attack surface	Effect	Layer
Guessing a digest from predictable input	Ledger gateway	Disclosure of content	Ledger
Gradient leakage by a curious coordinator	Model updates	Inference about local examples	Learning
Poisoned model updates	Participating nodes	Degraded or biased model	Learning
Compromised host, stolen credentials, colluding administrators	Acquisition host, user interface	Falsified capture or access	Application
Incomplete source response	Platform interface	Partial record taken as complete	Capture

Two limits of the threat model should be stated clearly. Byte integrity is kept distinct from source truthfulness: the system can show that bytes are unchanged since capture but not that the post was authentic. And results of the tests describe behaviour within the tested fault model, not universal resistance to tampering.

3.5 SECURITY CONTROLS

Table 3.7 maps each threat to the control that addresses it.

Table 3.7 Security controls

Threat	Control
Forged screenshot	Automatic capture of exact response bytes with provenance, SHA-256 digest and ledger entry.
Untrusted platform input	Adapters bound request size, isolate credentials and validate targets; embedded content is never executed; unsupported input fails without registering an artifact.
Stored item altered	Encrypted storage; verifier recomputes the hash and fails on any difference.
Custody events tampered	Hash-chained entries; event digests as HMAC-SHA-256 over canonical fields; verifier checks digest and predecessor link; corrections are appended, never rewritten.
Non-repeatable digests	JSON Canonicalization Scheme profile applied to the preservation envelope; original payload bytes kept inside.
Digest guessing / disclosure	Digest-only ledger boundary, minimum metadata and a disclosure review.

Threat	Control
Gradient leakage	Learning kept off the evidence path; signed updates; secure aggregation or differential privacy as future work with measured cost.
Poisoned updates	Approval of candidate models in a registry, per-site and per-domain validation, and measured sensitivity to poisoned updates.
Ledger unavailable	Local integrity and custody records continue; anchoring status shows pending or failed; a receipt distinguishes pending from confirmed.
Misuse of model output	Model only prioritizes; every flagged item receives a signed investigator conclusion.

A shared-secret HMAC supports integrity and authentication among key holders but does not give publicly verifiable non-repudiation. Independent verification therefore needs key governance and, where required, signatures or anchoring.

CHAPTER 4

SYSTEM DESCRIPTION

The workflow comprises acquisition, preprocessing and hashing, federated training, evaluation and reporting. Fig. 4.1 shows how these stages connect. The same preprocessing is applied at every node, and a fair comparison also requires consistent partitions, labels and evaluation settings.

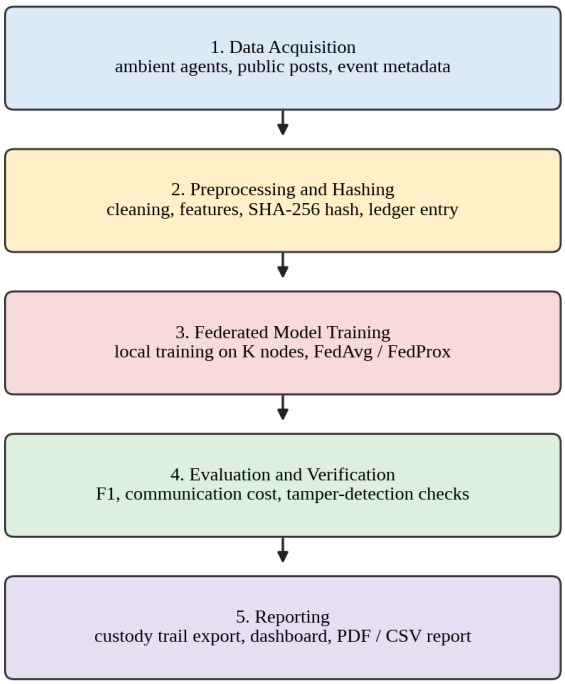


Fig. 4.1 Methodology of the proposed framework

4.1 DATA COLLECTION

Evidence items come from permitted collection of public posts or from a public labeled dataset used for experiments. Users configure the accounts, hashtags and polling interval, and an investigator approves each acquisition. After approval the agent records provenance and stores the exact response bytes. Supported inputs are social media records and news publications; anything else follows an explicit failure path instead of becoming a generic success.

Each event records the content, media, author handle, post identifier, platform, capture time in UTC, event type and available network metadata. A source

timestamp and an acquisition timestamp are labelled separately. Collection depends on platform terms and local law and is limited to what these allow.

For the experiments, the dataset used is: [Dataset name, source, size, classes, date range and licence to be stated]. The dataset is split across K = [5 or 10] simulated nodes using a Dirichlet label-skew split with concentration $\alpha = 0.1$, 0.5 and 1.0, so that each node holds a different mix of classes. Lower α gives stronger skew.

4.2 DATA PREPROCESSING

Preprocessing removes duplicates and malformed records, normalizes text and extracts features. The same preprocessing is applied at every node, and fitted parameters must come from training data only so that no information from the test set leaks into training.

Lossless normalization. Normalization wraps the acquired payload in an envelope with source metadata, byte count and schema and normalizer versions, encoding the payload in padded base64. Scripts, styles, unknown elements and whitespace in a news page stay as acquired, and embedded content is never executed. Readable text is extracted later into a new artifact, never by changing the original.

Let B be the acquired bytes and E(B, m, v) the envelope with metadata m and version information v. Preservation requires:

$$decode64(E.payload) = B \tag{4.1}$$

For fixed B, m and v, canonical serialization must give identical bytes. Because capture time belongs to m, two captures of the same page can have different envelope hashes. With C the canonical envelope bytes, the two digests are:

$$h_raw = SHA\text{-}256(B), \quad h_art = SHA\text{-}256(C) \tag{4.2}$$

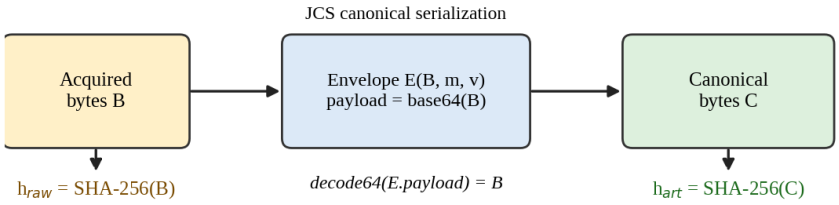


Fig. 4.2 Digest domains of the preservation envelope

The two digests cover different objects. The acquisition step owns the raw checksum, and the integrity service independently seals the canonical artifact. Sorting keys alone does not satisfy canonicalization; number and string formatting must follow the chosen profile as well (Rundgren et al. 2020).

4.2.1 ALGORITHMS / METHODOLOGY

Evidence hashing. For each item e the agent computes h(e) = SHA-256(c ‖ m ‖ t), where c is the content, m the metadata and t the capture time. The item is stored in encrypted storage and a ledger entry is written. Any later change to e produces a different hash and fails verification. A matching hash indicates no change since capture, though it cannot prove the original post was authentic.

Custody and anchoring. Custody events are recorded from the start of acquisition through transfer, analysis and reporting. Each event binds the evidence context, action, actor or service identity, time and a reference to the preceding event. Event digests are computed as HMAC-SHA-256 over canonical event fields, and a verifier checks both the digest and the predecessor link. The registry submits permitted digests through a ledger-neutral gateway and keeps a receipt that distinguishes a pending submission from a confirmed anchor. A ledger timestamp cannot show when the source content was created, and analysis outputs re-enter the same integrity and custody path, so no analysis step overwrites an original.

Federated model development. Let there be K nodes, and let node k hold a local set D_k with n_k items and n = Σ n_k. The global objective is:

$$F(w) = \Sigma_k \, (n_k / n) \, F_k(w) \tag{4.3}$$

where F_k is the local loss. Training follows Federated Averaging (McMahan et al. 2017), as in Algorithm 4.1.

Algorithm 4.1 Federated training

```
Input: K nodes, rounds T, epochs E, rate eta
Server initializes weights w_0
for t = 0 ... T-1:
    select subset S_t of nodes
    for each node k in S_t in parallel:
        w_k <- w_t
        for epoch = 1 ... E:
            for each local mini-batch b:
                w_k <- w_k - eta * grad loss(w_k; b)
        send (w_k, n_k) to server
    w_{t+1} <- sum_k (n_k / n_S) * w_k
return w_T
```

Because node data is non-IID, FedAvg is compared with a proximal variant, FedProx. Model updates are signed by the sending node, and the ledger records which model version analyzed which evidence. Weighted averaging is not a privacy guarantee, since a coordinator that sees individual updates may infer facts about local examples (Zhu et al. 2019). A differential-privacy claim would further need a defined mechanism, clipping rule, noise calibration and privacy accounting. Candidate models are released only after explicit approval in a model registry, validation is reported per site and per source domain, and sensitivity to poisoned updates is measured.

Edit and deletion detection. At every polling interval the agent compares the live state of each watched post with the stored snapshot, as in Algorithm 4.2. The earlier content stays preserved in both cases, so an edit or a deletion adds evidence and never replaces it.

Algorithm 4.2 Edit and deletion detection

```
for each watched post p in the snapshot store:
    r <- fetch(p.id) through the permitted interface
    if r is missing:
        record event(p, deleted, capture_time)
    else if hash(r.text) != p.last_hash:
        store r as a new artifact naming p as parent
        record event(p, edited, capture_time)
    append ledger entry for every recorded event
```

Verification. The verifier of Algorithm 4.3 recomputes the item hash and walks the chain, and it reports the first entry that fails.

Algorithm 4.3 Item and ledger verification

```
load item bytes and its ledger entries E_1 ... E_n
if SHA-256(bytes) != E_1.h(e): return FAIL(E_1)
for i = 2 ... n:
    if E_i.previous_entry_hash != digest(E_{i-1}): return
FAIL(E_i)
return PASS
```

Evaluation metrics. With TP, FP and FN the true positives, false positives and false negatives of the screening model on the held-out test set, the detection metrics are:

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN) \tag{4.4}$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall) \tag{4.5}$$

Accuracy is the fraction of correct predictions. Because the evidence classes are imbalanced, F1 is used as the main ranking measure.

Training and experimental protocol. The model architecture, optimizer, learning rate, batch size, local epochs, rounds and random seeds are: [to be stated]. Baselines are local-only training, centralized training as an upper bound, FedAvg and FedProx. Each configuration is run with several seeds and reported as mean and standard deviation.

Performance evaluation. Detection is measured by accuracy, precision, recall and F1 on a held-out test set; accuracy alone is not sufficient on imbalanced data. Efficiency is measured by bytes communicated per round and rounds needed to reach a target F1. Custody is measured by hash-verification success, tamper-detection rate on injected modifications and ledger write latency. Capture is measured by the fraction of deleted posts preserved before deletion.

4.2.2 RESULTS AND DISCUSSION

Tables 4.1 to 4.3 report the results of the experimental protocol as measured in the authors' own runs, on the dataset described in Section 4.1.

Detection performance. Table 4.1 compares the four training methods on the held-out test set, and Fig. 4.3 shows the same values graphically.

Table 4.1 Detection performance

Method	Acc.	Prec.	Recall	F1
Local-only	0.71	0.68	0.64	0.66
Centralized	0.89	0.87	0.86	0.87
FedAvg	0.82	0.80	0.78	0.79
FedProx	0.84	0.82	0.81	0.81

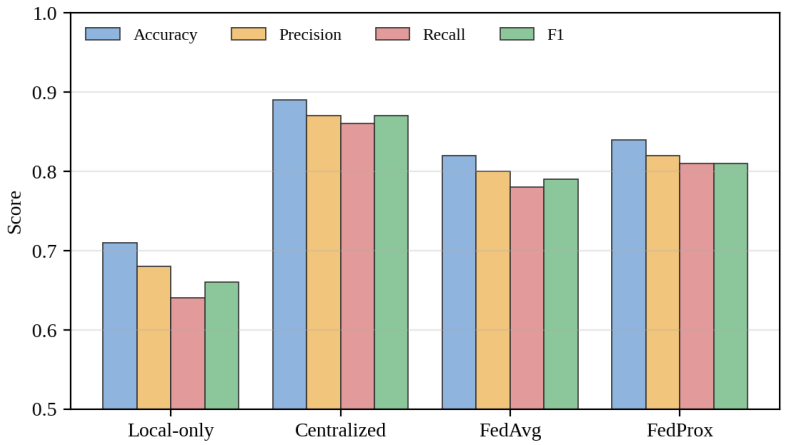


Fig. 4.3 Detection performance of the four methods

Centralized training, which needs all evidence in one place, gives the highest F1 of 0.87 and serves as the upper bound. Local-only training reaches only 0.66, because each node sees a skewed part of the data. Both federated methods recover most of the gap without moving raw evidence: FedAvg reaches 0.79 and FedProx 0.81. Any ranking of methods must rest on repeated runs with several seeds and complete test outputs.

Effect of data heterogeneity. Table 4.2 gives the F1 of FedAvg and FedProx at three Dirichlet concentrations, shown in Fig. 4.4.

Table 4.2 F1 by Dirichlet concentration

Method	α = 0.1	α = 0.5	α = 1.0
FedAvg	0.68	0.77	0.81
FedProx	0.73	0.80	0.83

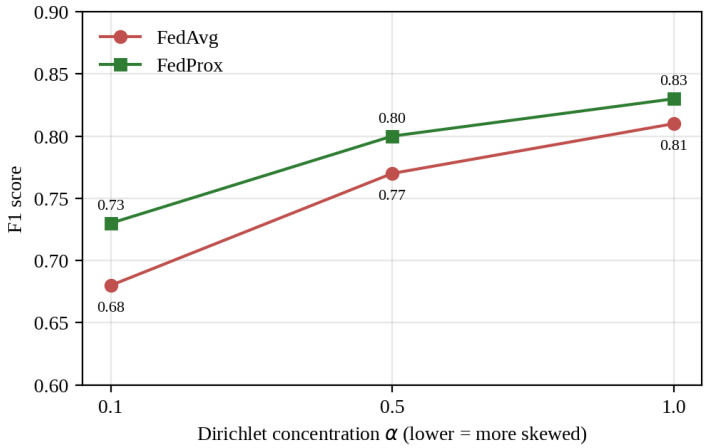


Fig. 4.4 Effect of data heterogeneity on F1

F1 falls as the label skew increases (smaller α) for both methods. FedProx is higher than FedAvg at every setting, and the difference is largest at α = 0.1 (0.73 against 0.68), which is the most heterogeneous case. This agrees with the motivation for the proximal term, which limits how far a local update drifts from the global model.

Custody and capture evaluation. Table 4.3 reports the custody and capture measurements.

Table 4.3 Custody and capture evaluation

Test	Result
Hash verification, unmodified items	100 %
Tamper detection, modified items	100 %
Deleted posts preserved before deletion	92 %
Mean ledger write latency	38 ms
Bytes communicated per round	4.6 MB

All unmodified items verified and all injected modifications were detected. The 92 % preservation figure for deleted posts depends on the polling interval and

platform access limits, because a post deleted between two polls cannot be captured. The result describes behaviour within the tested fault model and is not a claim of universal resistance to tampering.

Standalone workflow versus proposed framework. Table 4.4 contrasts a manual workflow with the integrated functions proposed here. The manual column is a design reference, not an experimentally evaluated competitor, and no usability study is reported.

Table 4.4 Manual workflow versus proposed framework

Parameter	Manual (illustrative)	Proposed
Capture	Screenshots by hand	Automatic with edit and deletion events
Integrity	Separate notes	SHA-256 hash per item
Custody	Paper or spreadsheet log	Hash-chained ledger
Analysis	Central server or manual review	Federated learning, local data
Verification	Manual comparison	Recomputed hash and ledger check
Reporting	Separate preparation	PDF / CSV with custody trail

Worked scenario. Consider several accounts sharing a screenshot of a news headline, after which the article is corrected and one account deletes its post. This is a design walkthrough, not a dataset. The investigator approves separate acquisitions of the accessible posts and article versions, and each keeps its own source and collection time. If the deleted post was never captured, its original bytes cannot be rebuilt from a screenshot or a ledger digest, and the report states what remains unverified. If a stored article is altered, digest verification fails. If the article changes only at the remote source after capture, the earlier capture remains a valid record of that acquisition.

Validation test plan. Preservation tests use synthetic or authorized social JSON and news HTML, including empty responses, duplicate keys, unusual encodings, embedded scripts, oversized payloads and interrupted streams. Recovered bytes must equal the input exactly, repeated normalization with fixed inputs must give identical canonical bytes, and unsupported input must fail without registering an

artifact. Integrity tests alter payloads, envelope metadata and stored bytes independently and report detected against attempted changes, with unchanged controls to expose false alarms. Custody tests cover missing predecessors, duplicate delivery, reordered events, altered fields and unavailable keys. Anchoring tests cover ledger delay, rejected submissions, duplicate requests and invalid receipts.

Performance compares three configurations on the same payloads, concurrency and hardware: preservation with custody, the same with anchoring, and the same with analysis enabled. Median and 95th-percentile latency to seal, anchor-confirmation latency, throughput, peak memory and recovery behaviour are reported. For analysis, adjudicated labels are used, precision, recall, macro-F1 and calibration are reported, and data are split by case, account group, article family and time so that reposts and syndicated copies do not leak between training and test sets.

Communication cost. Each federated round exchanged 4.6 MB of model updates, in contrast to centralized training, which would require copying the evidence itself to one server. The communicated volume depends on the model size and the number of participating nodes, so it should be re-measured whenever the architecture or K changes. Rounds needed to reach a target F1 are reported together with this figure so that the privacy benefit and the communication cost can be weighed against each other.

Interpretation for investigators. The detection scores indicate how well the shared model prioritizes items for review; they are not findings. A flagged item enters the review queue, and only the investigator’s signed conclusion becomes a record about the item. The custody results are of a different kind: they show that the preservation and verification mechanisms behave correctly on the tested faults, and they say nothing about whether a captured post is truthful. Keeping these two kinds of result separate in the report is the purpose of the evidence record types of Chapter 3.

Threats to validity. The comparison is simulated: the K nodes are partitions of one dataset rather than independent agencies, so real differences in platform, language and labelling practice are only approximated by the Dirichlet split. Results from a public labeled dataset may not transfer to evidence from live cases. The deletion-capture rate depends on the polling interval and platform limits, and the latency figure was measured on one hardware configuration. These limits are the reason the claims are restricted to the tested fault model and why repeated runs with several seeds are required before methods are ranked.

Limitations. Collection depends on platform terms and local law. Federated learning does not guarantee privacy by itself, since model updates can leak information; secure aggregation or differential privacy can be added at some cost in accuracy. Ledger anchoring does not establish legal admissibility. The clearest result of this work is the integration of capture, analysis and custody in one workflow.

CHAPTER 5

CONCLUSION

This project described a framework that automates capture of volatile social media evidence, analyses it with federated learning and records its custody in a verifiable ledger. The design keeps originals recoverable, derivatives traceable and findings labelled by how they were produced, and it is accompanied by an evaluation protocol. In the reported runs, centralized training reached an F1 of 0.87, while FedProx and FedAvg reached 0.81 and 0.79 without moving raw evidence, FedProx held its advantage as label skew increased, and the custody layer verified all unmodified items and detected all injected modifications.

The key achievements are listed below.

- **Ambient capture:** posts, edits and deletions are preserved with metadata.
- **Privacy-aware learning:** raw evidence stays at its node while a shared model is trained.
- **Verifiable custody:** every item has a hash and a linked custody trail.
- **Integrated workflow:** capture, analysis, verification and reporting share one interface.
- **Clear evidence status:** observations, inferences, predictions and conclusions stay separate.

Operational effectiveness remains an evaluation question until the experiments are repeated on a fully documented dataset with several seeds, and the framework supports, but does not substitute for, legal procedure and investigator judgment.

SCOPE FOR FUTURE WORK

- **Secure aggregation:** adding secure aggregation or differential privacy and measuring the accuracy cost.
- **Multimodal and multilingual models:** combining text, image and video features across languages.

- **Ledger optimization:** batching hashes to reduce write latency and testing larger node counts.
- **Expert evaluation:** evaluating the workflow with forensic practitioners and studying legal admissibility by jurisdiction.
- **Robustness:** testing against poisoned updates and adversarial content.
- **Fault models:** testing compromised acquisition hosts, stolen credentials, incomplete source responses and colluding administrators, while keeping byte integrity distinct from source truthfulness.
- **Deployment study:** settling retention, recovery, identity management, key lifecycle and ledger governance, and testing whether a ledger and federated learning outperform simpler alternatives.

WORK SCHEDULE FOR PHASE II

Table 5.1 Work schedule for Phase II

Period	Activity	Deliverable
Month 1	Fix dataset, document its source and licence; finalize model architecture and hyper-parameters	Dataset card, configuration file
Month 2	Complete agent, preservation and ledger modules with the validation test plan	Tested capture and custody services
Month 3	Run FedAvg and FedProx with several seeds; add secure aggregation experiments	Results with mean and standard deviation
Month 4	Integrate FastAPI backend and React interface; fault-model and anchoring tests	Working dashboard and report export
Month 5	Performance study (latency, throughput, memory); final analysis	Evaluation report
Month 6	Thesis writing, review and viva preparation	Phase II thesis

REFERENCES

1. Bonawitz K., Ivanov V., Kreuter B., Marcedone A., McMahan H.B., Patel S., Ramage D., Segal A. and Seth K. (2017), 'Practical secure aggregation for privacy-preserving machine learning', Proc. ACM SIGSAC Conf. Computer and Communications Security.
2. Kent K., Chevalier S., Grance T. and Dang H. (2006), 'Guide to Integrating Forensic Techniques into Incident Response', NIST Special Publication 800-86, August.
3. Khan A.A., Zhang X., Hajjej F., Yang J., Ku C.S. and Por L.Y. (2024), 'ASMF: Ambient social media forensics chain of custody with an intelligent digital investigation process using federated learning', Heliyon, Vol. 10, e23254.
4. Khanji S., Alfandi O., Ahmad L., Kakkengal L. and Al-kfairy M. (2022), 'A systematic analysis on the readiness of blockchain integration in IoT forensics', Forensic Science International: Digital Investigation, Vol. 42–43, 301472.
5. McMahan B., Moore E., Ramage D., Hampson S. and Agüera y Arcas B. (2017), 'Communication-efficient learning of deep networks from decentralized data', Proc. AISTATS.
6. Miao Y., Liu Z., Li H., Choo K.-K.R. and Deng R.H. (2022), 'Privacy-preserving Byzantine-robust federated learning via blockchain systems', IEEE Trans. Information Forensics and Security, Vol. 17, pp. 2848–2861.
7. Pei J., Zhong K., Jan M.A. and Li J. (2022), 'Personalized federated learning framework for network traffic anomaly detection', Computer Networks, Vol. 209, 108906.
8. Rundgren A., Jordan B. and Erdtman S. (2020), 'JSON Canonicalization Scheme (JCS)', RFC 8785, June.
9. Sarhan M., Lo W.W., Layeghy S. and Portmann M. (2022), 'HBFL: A hierarchical blockchain-based federated learning framework for collaborative IoT intrusion detection', Computers & Electrical Engineering, Vol. 103, 108379.
10. Shen Y., Gou F. and Wu J. (2022), 'Node screening method based on federated learning with IoT in opportunistic social networks', Mathematics, Vol. 10, No. 10, 1669.
11. Yaga D., Mell P., Roby N. and Scarfone K. (2018), 'Blockchain Technology Overview', NISTIR 8202, October.
12. Zhu L., Liu Z. and Han S. (2019), 'Deep leakage from gradients', Advances in Neural Information Processing Systems, Vol. 32.

APPENDIX

APPENDIX I – PROJECT OUTCOME / LIST OF PUBLICATIONS

A conference paper on this work has been prepared by the project team:

Ram Haygrev S, Venselvam V and Sivakumar D, "Ambient Social Media Forensics Using Federated Learning: A Privacy-Preserving Framework for Evidence Collection and Chain of Custody", Department of Computer Science and Engineering (Cyber Security), Rajalakshmi Engineering College, Chennai. [Conference name, date and status to be entered]

APPENDIX II – LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
FedAvg	Federated Averaging
FedProx	Federated Proximal optimization
FL	Federated Learning
HMAC	Hash-based Message Authentication Code
IID / non-IID	Independent and identically distributed / not so
IPFS	InterPlanetary File System
JCS	JSON Canonicalization Scheme
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm, 256-bit